

CCAF MASTER HANDBOOK

Claude Certified Architect Foundations (CCAR-F)

Complete Exam Preparation Guide - 2026 Edition

All 37 exam objectives | 11 modules | 40 scenario drills with answer key
Decision trees | Memory tables | Exam traps | Last-minute cheat sheet

Contents

How to Use This Handbook.....	5
Exam at a Glance.....	5
The Seven Domains (reconstructed from the score-report objectives)	5
Module 1 - Claude API Fundamentals.....	7
1.1 The Messages API	7
1.2 Stop Reasons (know all of them)	7
1.3 Context Window	8
1.4 Streaming (SSE)	8
1.5 Messages API vs Message Batches API (scored objective)	8
1.6 Common API Mistakes (frequent distractors)	9
Module 2 - Tool Use & Agentic Loops.....	10
2.1 The Tool Use Lifecycle.....	10
2.2 tool_choice - Controlling Whether Tools Are Used	10
2.3 Parallel Tool Calls in a Single Turn (scored objective).....	11
2.4 Sequencing Multi-Tool Workflows (scored objective).....	11
2.5 Designing Good Tools	12
2.6 Post-Tool-Use Reasoning	12
2.7 Stop Sequences in Agent Design.....	12
Module 3 - Structured Output & Extraction Pipelines.....	13
3.1 Choosing the Structured Output Method (scored objective)	13
3.2 Schema Design: Optional, Nullable, Enums (scored objective)	13
3.3 Extraction Accuracy Patterns (scored objective)	13
3.4 Inclusion / Exclusion Boundaries (scored objective).....	14
3.5 Handling Truncated Structured Output (scored objective)	14
3.6 Validation and Retry Loops	14
3.7 Human Review Routing (scored objective).....	15
Module 4 - Multi-Agent Systems	16
4.1 Why Subagents	16
4.2 Orchestration Patterns and When to Use Each (scored objective)	16
4.3 Delegation Style: Goal-Oriented vs Procedural (scored objective)	16
4.4 Self-Contained Subagent Prompts (scored objective)	17
4.5 Configuring Subagents: Tools, Context, System Prompt (scored objective)	17
4.6 Subagent Output Schemas (scored objective).....	18
4.7 Diagnosing Misconfigured Subagent Spawning (scored objective).....	18
4.8 Dynamic Task Decomposition (scored objective)	18
4.9 State Persistence and Resumption for Pipelines (scored objective)	19

4.10 Synthesis That Preserves Uncertainty (scored objective).....	19
4.11 Failure Handling: Partial Failures, Retries, Structured Errors	19
Module 5 - Context Management, RAG & Knowledge	20
5.1 What Fills the Context.....	20
5.2 Context Window Optimization Techniques (scored objective)	20
5.3 Externalizing Memory: Isolation, Scratchpads, Targeted Reads (scored objective).....	20
5.4 Session Resumption (scored objective)	21
5.5 Attention Dilution (the 'attention delusion' problem)	21
5.6 Summary Injection / Stale Context Problem	21
5.7 RAG vs Context vs Fine-Tuning	21
5.8 Systematic Codebase Exploration (scored objective).....	22
Module 6 - Claude Code.....	23
6.1 Selecting the Right Configuration Mechanism (scored objective).....	23
6.2 Skills Essentials	24
6.3 context: fork - Isolated Skill Execution (scored objective).....	24
6.4 Plan Mode vs Direct Execution vs Multi-Phase (scored objective).....	24
6.5 Headless Mode & CI/CD (scored objective).....	25
6.6 Automated Review Configuration (scored objective).....	26
6.7 Specialized Review Passes (scored objective).....	26
6.8 Reducing Review False Positives (scored objective).....	26
6.9 High-Quality Test Generation (scored objective)	26
6.10 Iterative Refinement Workflows (scored objective).....	26
Module 7 - Model Context Protocol (MCP)	28
7.1 Architecture	28
7.2 Resources vs Tools - the Scored Distinction	28
7.3 Integrating MCP Servers (scored objective)	28
7.4 Writing MCP Tool Descriptions (scored objective)	29
Module 8 - Prompt Engineering Fundamentals.....	30
Module 9 - Master Decision Trees & Memory Tables	31
9.1 API & Output Decisions.....	31
9.2 Agent Architecture Decisions.....	31
9.3 Claude Code Decisions	31
9.4 One-Line Memory Table (high-frequency answers)	31
Module 10 - Exam Traps (the classic wrong-vs-right pairs)	33
Module 11 - Rapid-Fire Scenario Drills (40 questions).....	34
Module 11 - Answer Key & Explanations.....	41
Appendix A - Full Objective Coverage Checklist (all 37 score-report objectives)	43
Appendix B - Last-Minute Cheat Sheet (read in 10 minutes)	45

Open in Microsoft Word and allow field updates (or press Ctrl+A then F9) to populate page numbers.

How to Use This Handbook

This handbook is built directly from the official CCAF score-report objectives (37 objectives, exam code CCAR-F). Every objective is covered in a dedicated section, and Appendix A maps each objective to the exact section that teaches it, so you can verify nothing is missed.

- **First pass (deep study):** read Modules 1 to 8 in order. Each section ends with a memory table and exam tips.
- **Second pass (drill):** work through Module 11 (Scenario Drills) without looking at answers, then review explanations.
- **Final 24 hours:** read Module 9 (Master Decision Trees), Module 10 (Exam Traps), and Appendix B (Last-Minute Cheat Sheet).
- **Weak-area repair:** use Appendix A to jump straight to any objective you scored low on.

EXAM TIP The CCAF is scenario-based. Almost every question describes a situation and asks you to **select the best approach**. Train yourself to spot the keyword in the scenario (bulk, must, interactive, isolated, read-only, runaway, changing data) - the keyword almost always determines the answer.

Exam at a Glance

Item	Detail
Exam name	Claude Certified Architect Foundations (CCAF)
Exam code	CCAR-F
Passing score	720 (scaled)
Question style	Scenario-based multiple choice; select the best architecture, configuration, or technique
Core products tested	Claude API (Messages, Batches, Tool Use), Claude Code, Claude Agent SDK subagents, MCP
Skills tested	Selecting the right mechanism, diagnosing misconfigurations, designing multi-agent and extraction pipelines

The Seven Domains (reconstructed from the score-report objectives)

#	Domain	What it covers	Handbook module
1	Claude API fundamentals	Messages vs Batches, stop reasons, context window, streaming	Module 1
2	Tool use & agentic loops	tool_choice, parallel tool calls, sequencing, loop mechanics	Module 2
3	Structured output & extraction	JSON schemas, enums/nullable, validation, truncation, review routing	Module 3

#	Domain	What it covers	Handbook module
4	Multi-agent systems	Orchestration patterns, subagent config, delegation, state persistence	Module 4
5	Context management	Optimization techniques, resumption, scratchpads, RAG vs context	Module 5
6	Claude Code	Config mechanisms, plan mode, headless CI/CD, reviews, context: fork	Module 6
7	MCP	Resources vs tools, scopes, auth, tool descriptions	Module 7

Module 8 covers prompt engineering fundamentals that appear inside scenarios across all domains.

Module 1 - Claude API Fundamentals

1.1 The Messages API

The Messages API (`POST /v1/messages`) is the synchronous, request-response way to talk to Claude. You send a list of `messages` (alternating `user` and `assistant` roles), an optional `system` prompt, and required `model` and `max_tokens`. Claude returns one assistant message.

```
POST /v1/messages
{
  "model": "claude-sonnet-4-5",
  "max_tokens": 1024,
  "system": "You are a precise data-extraction assistant.",
  "messages": [
    { "role": "user", "content": "Extract the invoice total." }
  ]
}
```

- **system** - instructions that frame the whole conversation (role, rules, output format). Not a message role; it is a top-level parameter.
- **max_tokens** - a hard cap on OUTPUT tokens only. If the answer needs more, generation stops with `stop_reason: max_tokens` (truncation).
- **temperature** - randomness. Use low (0 to 0.3) for extraction, classification, and code; higher (0.7 to 1.0) for creative work.
- **Multi-turn** - the API is stateless. You must resend the full conversation history on every call; Claude remembers nothing between requests.

TRAP Stateless API means: if a scenario says the model 'forgot' earlier turns, the bug is that the application did not resend history - not a model setting.

1.2 Stop Reasons (know all of them)

stop_reason	Meaning	Correct application response
end_turn	Claude finished naturally	Show/parse the answer; turn is complete
tool_use	Claude wants to call a tool	Execute the tool(s), return tool_result, call the API again
max_tokens	Output hit the max_tokens cap	Response is TRUNCATED - raise the cap or split the task; never parse as complete
stop_sequence	A custom stop string was generated	Generation stopped at your delimiter; the stop string is not included in output
pause_turn	A long-running server tool paused	Send the response back as-is to continue the turn
refusal	Claude declined the request	Handle gracefully; do not blind-retry the identical prompt

REMEMBER `tool_use` = 'do something and come back'. `max_tokens` = 'I was cut off'. `end_turn` = 'I am done'. These three drive almost every loop-control question.

1.3 Context Window

The context window is everything Claude can see in one request: system prompt + full conversation history + tool results + attached documents + the current message. Standard windows are large (200K tokens on current models, with larger options on some models), but they are finite and everything competes for space and attention.

- Input tokens + output tokens must fit inside the window.
- Long histories cost more, respond slower, and can dilute attention (see Module 5, Attention Dilution).
- When you exceed the window you must drop, summarize, or externalize older content - the API will not silently do it for you.

1.4 Streaming (SSE)

With `"stream": true`, the API returns Server-Sent Events so tokens arrive incrementally (`message_start`, `content_block_delta`, `message_stop`, and related events). Streaming improves PERCEIVED latency for interactive UIs; it does not make total generation faster or cheaper.

EXAM TIP Scenario says 'users watch the answer appear' or 'chat UI feels slow to start' - answer: streaming. Scenario says 'reduce total cost/latency of 10,000 jobs' - streaming is a distractor; the answer is Batches or parallel tool calls.

1.5 Messages API vs Message Batches API (scored objective)

The Message Batches API submits many independent requests as one asynchronous batch. You poll (or check back) for results instead of blocking. Batches cost **50% less** than the same requests sent synchronously, support very large jobs (up to 100,000 requests or 256 MB per batch), typically finish in well under an hour, and are guaranteed to complete within 24 hours.

Signal in the scenario	Choose	Why
User is waiting / interactive chat / real-time	Messages API (sync)	Latency matters; blocking is fine
Agentic loop where each step depends on the last	Messages API (sync)	Steps are sequential and interactive
Nightly job, 50,000 documents, results needed by morning	Message Batches API	Bulk, non-blocking, 24h window acceptable
Cost reduction on high-volume offline classification	Message Batches API	50% discount on batch processing
Pipeline must not block a web request thread	Message Batches API	Asynchronous by design
A few seconds of latency acceptable, but need answer in THIS request	Messages API (sync)	Batches give no per-request latency guarantee

REMEMBER Decision in one line: **Interactive or dependent = Messages. Bulk, offline, or cost-driven = Batches.**

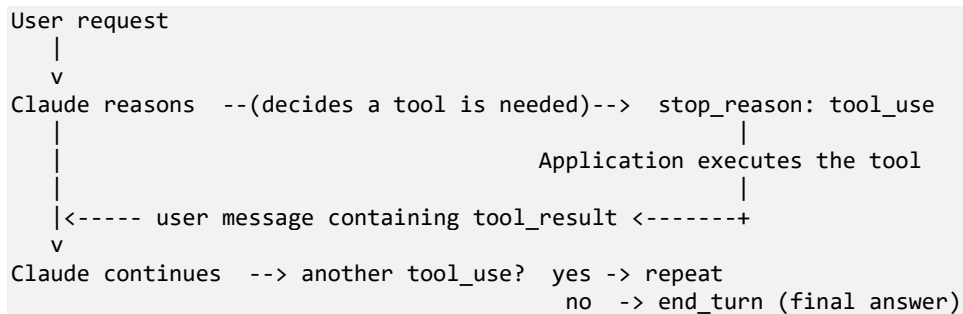
TRAP Do not pick Batches just because volume is 'high' - if each request's answer is needed immediately in a user-facing flow, Batches is wrong even at high volume.

1.6 Common API Mistakes (frequent distractors)

- Parsing a `max_tokens`-truncated response as if it were complete JSON.
- Forgetting to resend conversation history (stateless API).
- Putting per-request instructions in `system` but expecting them to change per turn without resending.
- Raising `max_tokens` to astronomical values instead of splitting the task (see Module 3.5).
- Treating streaming as a cost or throughput optimization.

Module 2 - Tool Use & Agentic Loops

2.1 The Tool Use Lifecycle



- You declare tools with **name**, **description**, and **input_schema** (JSON Schema).
- **Claude never executes anything**. It emits a **tool_use** content block (with a unique **id** and **input**); YOUR application runs the tool and returns a **tool_result** block referencing that **tool_use_id**.
- The **tool_result** goes back inside the **next user message**, then you call the API again with the full history.
- This request-execute-return cycle repeated until **end_turn** is the **agentic loop**.

TRAP Every **tool_use** block must get a matching **tool_result** in the IMMEDIATELY FOLLOWING user message. Skipping one, or spreading results across several messages, is an API error - a classic 'what is wrong with this code' question.

2.2 tool_choice - Controlling Whether Tools Are Used

tool_choice	Behavior	Use when
{"type": "auto"}	Claude decides whether to use any tool (default when tools are provided)	Tools are optional helpers
{"type": "any"}	Claude MUST use some tool (cannot answer in plain text)	Every response must be a tool call, tool selection varies
{"type": "tool", "name": "X"}	Claude MUST call tool X specifically	Guaranteed structured output through one known tool
{"type": "none"}	Claude cannot use tools this turn	Force a plain-text answer while tools remain defined

The scored pattern: 'a conversational (plain-text) response would break the downstream parser - guarantee tool invocation.' The answer is **any** (when several tools are valid) or **tool** with a name (when one specific schema is required). Relying on prompt wording alone ('always respond with the tool') is the wrong answer - it is hope, not enforcement.

REMEMBER Memory table: Unknown/varies which tool -> **any**. Known single tool -> **tool + name**. Optional -> **auto**. Forbidden -> **none**.

EXAM TIP Nuance worth knowing: forcing tool use (`any/tool`) suppresses free-text chain-of-thought before the call, and extended thinking only works with `auto/none`. If a scenario needs visible reasoning AND a tool call, that is a trade-off question.

2.3 Parallel Tool Calls in a Single Turn (scored objective)

Claude can emit **multiple `tool\_use` blocks in one assistant response**. This is the mechanism for running independent subtasks in parallel and cutting round-trip latency: one model turn fans out N tool calls, your application executes them **concurrently**, and you return ALL results together.

```
Assistant turn:
  tool_use #1  get_weather(city="Pune")
  tool_use #2  get_weather(city="Delhi")
  tool_use #3  get_flight_price(from="AMD", to="DEL")

Application: run all three CONCURRENTLY (Promise.all / asyncio.gather)

Next user message (ONE message):
  tool_result for id #1
  tool_result for id #2
  tool_result for id #3
```

- **How to enable/encourage:** it works out of the box with `tool_choice: auto`; strengthen it by prompting, e.g. 'For maximum efficiency, whenever operations are independent, invoke all relevant tools simultaneously rather than sequentially.'
- **How to disable:** set `disable_parallel_tool_use: true` inside `tool_choice` when calls must be one-at-a-time.
- **Rule:** all `tool_result` blocks for that turn go in a SINGLE user message, each matching its `tool_use_id`.
- **When NOT to parallelize:** when call B needs call A's output (dependent calls) - those must be sequential turns.

REMEMBER Parallel tool calls reduce **round trips** (latency), not token cost per call.
Independent = parallel in one turn. Dependent = sequential turns.

TRAP 'The agent makes 5 independent lookups and users complain about latency' - the answer is parallel tool calls in one turn, NOT batches, NOT streaming, NOT a bigger model.

2.4 Sequencing Multi-Tool Workflows (scored objective)

When tools depend on each other (look up the customer ID before fetching orders), the workflow must obtain **prerequisite data before dependent tools are called**. Three levers, strongest last:

1. **Describe dependencies in tool descriptions** ('requires a customer_id obtained from search_customer') - helpful, but advisory only.
2. **Prompt the ordering** ('First call search_customer, then get_orders') - better, still not guaranteed.
3. **Enforce programmatically** - the application only exposes/permits the dependent tool after the prerequisite result exists, or validates inputs and returns an error `tool_result` ('call search_customer first') that steers Claude back on track.

TRAP Exam trap pattern: 'improve the tool description' vs 'enforce the workflow in the application'. When the scenario says a step **MUST** happen or **MUST** happen first, description tweaks are the wrong answer - **programmatic/workflow enforcement** is right.

2.5 Designing Good Tools

- **Names:** verb_noun, unambiguous (`get_invoice_by_id`, not `data2`).
- **Descriptions:** the single highest-leverage factor. State what it does, when to use it, when NOT to use it, input formats, and how it differs from similar tools (full guidance in Module 7.4, which applies equally here).
- **Schemas:** typed parameters with `enum` for closed sets, `required` only for truly required fields, and descriptions on every property.
- **Granularity:** one clear job per tool. A mega-tool with a `mode` flag increases wrong calls.
- **Error results:** return structured, instructive errors as `tool_result` with `is_error: true` so Claude can self-correct.

2.6 Post-Tool-Use Reasoning

After a `tool_result` arrives, Claude decides the next step: answer the user, call another tool, or ask for clarification. Design your loop so this decision point is visible: log each turn's `stop_reason`, cap iterations (max-turns guard), and detect repeated identical tool calls (a loop symptom).

Loop symptom	Likely cause	Fix
Same tool called repeatedly with same input	Tool result unhelpful or malformed	Return structured results/errors the model can act on
Loop never ends	No termination guard	Cap turns programmatically; instruct 'answer when you have enough'
Claude answers without calling a required tool	<code>tool_choice</code> : auto + weak prompt	<code>tool_choice</code> : any / specific tool
API 400 after tool turn	Missing/misplaced <code>tool_result</code>	One user message with all <code>tool_results</code> , matching ids

2.7 Stop Sequences in Agent Design

`stop_sequences` are custom strings that end generation the moment Claude emits them (`stop_reason: stop_sequence`; the sequence itself is excluded from output). Use them to stop at a delimiter (e.g. generate one item then stop at `\n\n`), to cut off before a section you do not want, or as a cheap output-format guard. They are a text-level control - for structured output guarantees, prefer tool use (Module 3).

Module 3 - Structured Output & Extraction Pipelines

3.1 Choosing the Structured Output Method (scored objective)

Method	How it works	Reliability	Choose when
Tool use with JSON Schema	Define a tool whose input_schema IS your output schema; force it with tool_choice	Highest - schema-enforced	Downstream code parses the output; compliance must be strict
Prefilled response	Start the assistant message yourself (e.g. prefill '{' or '<result>')	Medium-high - strong steer, no schema guarantee	Light structure, skip preambles (note: incompatible with extended thinking)
Prompt-based formatting	Ask for JSON/XML in the instructions, optionally with examples	Lowest - can drift, add prose, or malform	Human-readable output, flexible structure acceptable

REMEMBER Strictness ladder: **prompt < prefill < tool use with schema**. 'Downstream system breaks on malformed output' always points to tool use + forced tool_choice.

3.2 Schema Design: Optional, Nullable, Enums (scored objective)

The goal of extraction schema design is to let the model **accurately represent missing or ambiguous information instead of fabricating values**.

- **Optional fields:** only truly-always-present fields go in **required**. If **invoice_number** is required but absent from the document, the model is cornered into inventing one.
- **Nullable values:** allow **"type": ["string", "null"]** (or an explicit null option) so 'not found' has a legal representation.
- **Enums:** use **enum** for closed sets (**["paid", "pending", "overdue"]**) to eliminate spelling drift - and include an **"unknown"/"other"** member so ambiguity does not get force-fitted into a wrong category.
- **Field descriptions:** each property gets a description, expected format, and an example ('date in YYYY-MM-DD').
- **Confidence fields:** add **confidence** (0-1) or per-field flags when a human-review pipeline consumes the output (see 3.7).

TRAP 'The model invents values for missing fields' - the fix is schema design (optional + nullable + unknown enum member), not 'tell it to try harder' and not raising temperature. Fabrication is usually a cornered-schema problem.

3.3 Extraction Accuracy Patterns (scored objective)

For varied, messy document formats, combine three patterns:

1. **Structured schema with optional fields** (3.2) - the legal way to say 'absent'.

2. **Format normalization instructions** - explicitly state canonical formats: dates to YYYY-MM-DD, amounts as numbers without currency symbols, phone numbers digits-only. Documents vary; the output must not.
3. **Few-shot examples** - 2 to 5 worked examples covering the tricky variants (a scanned layout, a missing field, an ambiguous value mapped to null/unknown). Examples teach edge-case behavior better than rules.

3.4 Inclusion / Exclusion Boundaries (scored objective)

Define **explicit boundaries** in the prompt so the model does not generate findings or extractions in categories where it is unreliable:

- **Inclusion:** 'Report ONLY: SQL injection, XSS, hard-coded credentials.'
- **Exclusion:** 'Do NOT report: style issues, performance guesses, anything in generated files or vendor directories.'
- **Unreliable categories:** if licensing-analysis findings are wrong 40% of the time, EXCLUDE the category rather than shipping noise - constraining scope beats hedging language.

EXAM TIP Scenario: 'the review tool produces plausible but wrong findings in category X' - the answer is to define exclusion boundaries (stop generating category X), not more few-shot examples for X.

3.5 Handling Truncated Structured Output (scored objective)

Symptom: large review/extraction responses end with `stop_reason: max_tokens` and the JSON is cut off mid-object. The exam-correct fix:

- **Split the task into smaller scoped calls** (per file, per section, per page, per category) so each response comfortably fits, then **merge the resulting data structures** in application code.
- **Why not just raise max_tokens?** Beyond practical limits it increases cost and latency, still eventually truncates, and very long single generations degrade quality. 'Increase max_tokens to 200,000' is nearly always the distractor.

REMEMBER Big output = **split and merge**, not 'more tokens'. (Same philosophy as the context trap: better decomposition beats bigger context.)

3.6 Validation and Retry Loops

1. **Schema validation** - parse and validate (e.g. Pydantic / JSON Schema validator). Catches shape errors.
2. **Business validation** - domain rules: total = sum of line items, dates not in the future, status transitions legal.
3. **Cross-field validation** - relationships between fields (end_date after start_date; currency matches country).
4. **Validation retry** - on failure, re-prompt WITH the specific validation errors included ('amount_total must equal sum of items; you returned 90 vs 100') so the model can correct itself. Bounded retries (2-3), then route to human review.

TRAP Classic trap: 'the model promises it will be accurate' vs programmatic checks. **Confidence is not enforcement** - validators, schemas, and retries are.

3.7 Human Review Routing (scored objective)

In high-volume extraction, you cannot review everything - and reviewing a RANDOM sample wastes reviewer time on easy documents while errors ship. Route to human review based on signals:

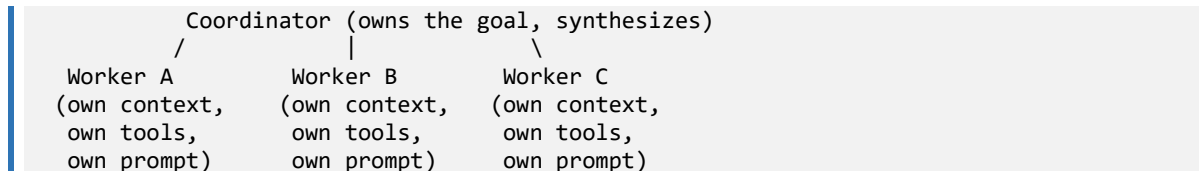
Routing signal	Example	Action
Model confidence score	Field confidence < 0.8, or overall confidence low	Send to review queue
Document characteristics	Poor scan quality, handwriting, unusual layout, new vendor template	Send to review queue
Field-level ambiguity	Nulls in critical fields, conflicting candidate values, 'unknown' enum chosen	Review just those fields
Business impact	Amount above threshold, regulated document type	Mandatory review regardless of confidence
High-confidence output	Everything confident and validated	Auto-accept; small calibration sample only

REMEMBER Confidence-based routing beats random sampling. Design the schema to EMIT the routing signals (confidence, nulls, flags) - routing quality is a schema-design decision.

Module 4 - Multi-Agent Systems

4.1 Why Subagents

A **subagent** is a separate Claude instance spawned by a coordinator (orchestrator) to do a scoped piece of work in its **own isolated context window**, with its **own system prompt and tool set**, returning only its result to the coordinator. Benefits: context isolation (worker output does not pollute the main thread), parallelism, specialization, and least-privilege tooling.



4.2 Orchestration Patterns and When to Use Each (scored objective)

Pattern	Shape	Strengths	Weaknesses	Choose when the scenario stresses...
Coordinator-worker	Central orchestrator dynamically spawns specialized workers and synthesizes	Adaptive coverage; lean coordinator context; central quality control	More total tokens; coordinator is a bottleneck/single point of failure	Research COVERAGE, heterogeneous subtasks, adaptive delegation
Parallel execution	Fan out independent subtasks simultaneously, fan in results	Lowest wall-clock LATENCY for independent work	Requires true independence; possible duplicated effort; needs a merge step	LATENCY with independent subtasks (e.g. search 5 sources at once)
Sequential pipeline	Stage A output feeds Stage B feeds Stage C	Handles dependencies; per-stage checkpoints and quality gates; easy resumption	Latency is additive; one stage stalls the pipeline	RELIABILITY, dependent stages, transformations, auditability

REMEMBER Keyword mapping: **coverage/breadth** -> **coordinator-worker (with parallel workers)**. **speed/independent** -> **parallel**. **dependent/checkpointed/reliable** -> **sequential pipeline**. Hybrids are normal: a coordinator that fans out parallel workers, then a sequential synthesis stage.

4.3 Delegation Style: Goal-Oriented vs Procedural (scored objective)

Style	You give the subagent	Enables	Risk	Use when
Goal-oriented	The outcome, constraints, and required report format ('Find all payment-related security issues in /billing; report	Adaptive behavior - the worker chooses its own path as it discovers things	Less predictable path	Exploration, research, debugging - the default for capable agents

Style	You give the subagent	Enables	Risk	Use when
	as JSON with file, line, severity')			
Procedural	Exact steps to execute in order	Determinism, compliance, repeatability	Cannot adapt when reality differs from the script; brittle	Regulated/compliance steps, known-good runbooks

Maintaining **coordinator visibility and control** with goal-oriented delegation: require a structured output contract (4.6), progress checkpoints for long tasks, and scoped tools - freedom in the HOW, strict contract on the WHAT.

4.4 Self-Contained Subagent Prompts (scored objective)

Subagents start with an **empty context** - they see none of the coordinator's conversation. A subagent prompt must therefore carry everything needed to finish without coming back for missing context:

- **The goal and scope** - precise task, boundaries, definition of done.
- **All findings so far** that the task depends on ('We already established the leak is in the image-processing service').
- **Structured data** it must use - IDs, file paths, config values, the actual records (not 'the list from earlier', which it cannot see).
- **Source metadata** - where each fact came from (URL, file, timestamp) so the worker can cite and extend it.
- **Constraints and the exact output schema** it must return.

TRAP 'The subagent keeps asking the coordinator for information' or 'results ignore earlier findings' = the prompt was not self-contained. The fix is packing context INTO the delegation prompt - not giving the subagent more tools, and not more turns.

4.5 Configuring Subagents: Tools, Context, System Prompt (scored objective)

Each subagent invocation is configured on three axes so the agent is **constrained to its designated role**:

1. **Tool restrictions** - grant ONLY the tools the role needs. A research subagent gets read/search tools; it does not get Write or Bash. Fewer tools = simpler decisions, fewer wrong calls, smaller blast radius (least privilege).
2. **Context scoping** - pass only the relevant slice of data (the one module, the one document set). Smaller focused context = better attention and cheaper runs.
3. **Role system prompt** - a dedicated system prompt defining the role, method, boundaries, and output contract ('You are a security reviewer. Only report...').

In the **Claude Agent SDK**, this is expressed as an **AgentDefinition** registered with the coordinator, e.g.:

```
agents: {
  "security-reviewer": {
```

```

description: "Reviews code changes for security vulnerabilities. Use for any auth,
payment, or input-handling change.",
prompt: "You are a security reviewer. Examine only ... Report JSON: {file, line,
severity, issue, fix}",
tools: ["Read", "Grep", "Glob"], // least privilege - no Write, no Bash
model: "sonnet"
}
}

```

4.6 Subagent Output Schemas (scored objective)

Design each worker's output contract around what **downstream synthesis and reporting** needs - a raw prose dump forces the synthesizer to re-parse everything; a bare JSON blob loses narrative context. The strong pattern combines three layers:

- **Structured data** (JSON matching a schema) - the machine-mergeable facts: findings list, fields, scores.
- **Prose summary** (2-5 sentences) - the narrative the synthesis agent can quote or weave into the report.
- **Citation metadata** - per-finding source (URL/file path), timestamp/version, and confidence, so the final report can attribute claims and flag stale or weak evidence.

REMEMBER Ask: 'what will the SYNTHESIZER do with this?' Merge -> structured. Narrate -> prose. Attribute/verify -> citations. The right answer usually includes all three.

4.7 Diagnosing Misconfigured Subagent Spawning (scored objective)

Symptom	Root cause	Fix
Worker 'cannot' perform its core action (e.g. cannot read files)	Missing tool permissions in its allowed tool list	Add the required tool to the subagent's tools
Worker is never selected / wrong worker selected	Incorrect AgentDefinition parameters - vague or overlapping description	Rewrite description: purpose + when to use + when not to use
Worker behaves generically, ignores role	Missing/weak role prompt in the AgentDefinition	Provide the dedicated system prompt and output contract
Coordinator never spawns anyone	Absent coordinator-to-subagent wiring - agents not registered, or coordinator lacks the delegation (Task) capability	Register AgentDefinitions with the coordinator; enable its delegation tool
Worker does out-of-role actions	Over-broad tool grant	Restrict tools to the role (least privilege)

4.8 Dynamic Task Decomposition (scored objective)

Good orchestrators **generate subtasks dynamically as information is discovered**, instead of executing a fixed sequence regardless of findings. Pattern: decompose an initial wave -> workers return findings -> coordinator RE-PLANS -> spawns the next wave targeted at what was actually found (e.g. discovery of a second auth path spawns a new worker for it; an empty result cancels

dependent subtasks). A fixed pre-planned sequence wastes work on dead ends and misses discoveries - it is the distractor answer.

4.9 State Persistence and Resumption for Pipelines (scored objective)

Multi-agent pipelines fail mid-run (timeouts, crashes, rate limits). Design so a restart neither repeats completed work nor loses prior findings:

- **Checkpoint after every completed stage/worker** to durable storage (files, DB): task id, status, inputs, and the FULL findings, not just 'done'.
- **A persistent task ledger** (e.g. a state JSON or scratchpad file) listing all subtasks with status: pending / running / complete / failed.
- **Idempotent stages** - re-running a completed stage is a no-op because the runner checks the ledger first.
- **On resume:** load the ledger + saved findings, inject them into the coordinator's context, and continue from the first incomplete subtask.

TRAP 'Keep everything in the coordinator's context window' is the wrong persistence answer - context dies with the process. Durable external state (files/DB) is the correct one.

4.10 Synthesis That Preserves Uncertainty (scored objective)

The synthesis agent that merges worker findings must **preserve source-level uncertainty** rather than collapsing conflicting data into single confident statements:

- Distinguish **well-established** findings (multiple independent sources agree) from **contested** ones (sources conflict) and **single-source** claims.
- Attribute: 'Source A (2026 filing) reports X; Source B reports Y - the discrepancy is unresolved.'
- Never average or silently pick a side when sources conflict; surface the conflict and, if useful, the likely reason.
- Carry worker confidence and citation metadata (4.6) into the final report.

4.11 Failure Handling: Partial Failures, Retries, Structured Errors

- **Partial failure:** 4 of 5 workers succeed - synthesize the 4, explicitly note the gap, retry or degrade gracefully. Do not discard the whole run.
- **Retries:** bounded (2-3) with backoff; retry **WITH** the error context so the second attempt differs from the first.
- **Structured error context:** workers report failures as data ({subtask, error_type, detail, partial_results}) so the coordinator can decide - retry, reroute, or proceed without.

Module 5 - Context Management, RAG & Knowledge

5.1 What Fills the Context

```
System prompt
+ Conversation history
+ Tool results
+ Retrieved documents (RAG)
+ Current user message
= everything competing for the context window
```

5.2 Context Window Optimization Techniques (scored objective)

As conversations exceed practical token limits, response quality drops before you even hit the hard limit. Four techniques - know each by its trade-off:

Technique	How it works	Trade-off / when to use
Summarization	Compress older turns into a compact summary that replaces them	Lossy - keep pointers to originals; best for long narrative history
Sliding window	Keep only the last N turns (plus pinned system/critical facts); drop the oldest	Simple and cheap; loses old details entirely - pair with pinned state
Structured state objects	Extract task state into a compact JSON (goal, decisions, open items, key facts) carried forward instead of raw transcript	Best signal-per-token for agent workflows; requires discipline to update
Selective retention	Keep decisions, constraints, and conclusions; drop verbose raw tool outputs after extracting what mattered	Targets the biggest token hogs (tool dumps) with least information loss

REMEMBER The four exam words: **summarize, slide, structure, select**. If the scenario mentions huge raw tool outputs -> selective retention. Long chat history -> summarization or sliding window. Agent needs durable working memory -> structured state object.

5.3 Externalizing Memory: Isolation, Scratchpads, Targeted Reads (scored objective)

- **Subagent isolation** - push token-heavy exploration into subagents; only conclusions return to the main context (Module 4.1, and `context: fork` in Module 6.6).
- **Scratchpad files** - the agent writes notes, findings, and plans to a file (`notes.md`, `state.json`) as it works. The file is durable external memory: survives context limits AND session ends, and can be re-read selectively.
- **Targeted file reading** - re-read only the specific files/sections needed now, instead of holding an entire codebase in context.

Together these sustain coherent codebase exploration across sessions that exceed context limits: explore -> write findings to scratchpad -> context fills -> new session reads the scratchpad -> continues.

5.4 Session Resumption (scored objective)

Restoring agent state accurately **without repeating prior work**:

1. **Persist state during the session** - scratchpad/state file with findings, decisions, and remaining plan.
2. **On resume, inject context** - load the summary/state file into the new session's context.
3. **Targeted re-analysis of CHANGED files only** - use `git diff` / timestamps to find what changed since the last session and re-read only those, instead of re-exploring everything.
4. **Validate stale assumptions** - re-check facts likely to have moved (branch state, dependencies) before acting on them.

5.5 Attention Dilution (the 'attention delusion' problem)

A large context does not mean everything in it is attended to equally. Overstuffed contexts bury the needle: critical facts in the middle of 150K tokens of noise get missed, and irrelevant content actively degrades answers. Fixes: retrieve only what is relevant (RAG), summarize, and structure the context (clear sections, the important thing stated near the task).

TRAP Exam trap: 'just use a bigger context window' vs **better decomposition/retrieval**. Bigger context is the distractor; curated context wins.

5.6 Summary Injection / Stale Context Problem

When summaries are injected across sessions, facts can be **stale**: 'the bug is in auth.py' may be false after yesterday's fix, but the model treats injected context as current truth. Mitigations: timestamp injected facts, mark confidence ('as of July 18'), and **revalidate important facts** (re-run the failing test, re-read the file) before acting on them.

5.7 RAG vs Context vs Fine-Tuning

RAG (Retrieval-Augmented Generation): the application searches a knowledge base, retrieves only the relevant chunks, and places them in the context. RAG does not replace context - **it builds the context**. (Tool-based retrieval via MCP is the same idea: fetch on demand instead of preloading.)

Scenario signal	Answer	Why
Millions of records / 100K support articles	RAG	Cannot and should not fit in context
Data changes hourly (prices, policies)	RAG	Update the knowledge base; effective immediately
General knowledge ('what is recursion')	Neither - just ask	Model already knows; retrieval adds nothing
Small doc already in the conversation	Plain context	It is already there
Teach a consistent STYLE/behavior	Fine-tuning	Behavior, not facts
New refund policy released today	RAG	Fine-tuning would require retraining for every change

REMEMBER RAG = changing or large FACTS. Fine-tuning = stable BEHAVIOR/style. Neither = general knowledge.

5.8 Systematic Codebase Exploration (scored objective)

Build **incremental understanding while managing the context window** - broad to narrow, taking notes:

1. **Glob** to map structure by filename pattern (`src/**/*.service.ts`) - cheap orientation.
2. **Grep** to locate content (`grep -r "processPayment"`) - find where things live without reading everything.
3. **Read** only the targeted files/sections that matter - the expensive step, spent surgically.
4. **Bash** to interrogate behavior (run tests, `git log`, build) when static reading is not enough.
5. Record findings in a **scratchpad** as you go so nothing must be re-derived (5.3).

Need	Tool
Find files by NAME/pattern	Glob
Find CONTENT across files	Grep
Inspect a SPECIFIC file	Read
EXECUTE something (tests, git, build)	Bash

TRAP Reading entire directories into context 'to be safe' is the anti-pattern answer. Search first (Glob/Grep), read second, and only what is needed.

Module 6 - Claude Code

6.1 Selecting the Right Configuration Mechanism (scored objective)

Claude Code has several ways to shape behavior. The exam tests whether you can pick the right one based on the guidance type and when it should apply:

Mechanism	What it is	When it applies	Use it for
CLAUDE.md	Markdown file at project root, loaded into context at session start and kept all session	Always, every session	Stable project FACTS: build commands, repo layout, conventions, team norms
.claude/rules/*.md with glob patterns	Rule files whose frontmatter globs scope them to matching paths	Only when matching files are touched	Path-scoped constraints ('in src/api/** always validate input'); an unscoped rule behaves like CLAUDE.md
Skills (.claude/skills/*/SKILL.md)	Reusable step-by-step procedures, loaded on demand (progressive disclosure); also exposed as /slash-commands	When invoked (by you or auto-triggered by description match)	PROCEDURES and workflows: deployments, release checklists, review flows
Hooks	Deterministic shell commands on lifecycle events (PreToolUse, PostToolUse, ...); PreToolUse can BLOCK a call (exit code 2)	Automatically at the event, every time	GUARANTEES and automation: block dangerous commands, auto-format after edits
Settings permissions (settings.json)	allow / ask / deny rules per tool and pattern; deny takes precedence; managed (admin) settings cannot be overridden	Enforced on every tool call	Access control: what tools/paths/commands are ever permitted

REMEMBER One-line router: **fact** -> **CLAUDE.md**; **scoped fact** -> **rule + glob**; **procedure** -> **Skill**; **must-happen/must-never-happen** -> **hook or permission**; **org-wide guarantee** -> **managed settings**.

TRAP 'Never do X' written in CLAUDE.md is the trap answer. Instructions are followed MOST of the time; under pressure, long sessions, or prompt injection they can fail. A real guardrail must be deterministic: a PreToolUse hook or a permissions deny rule. Likewise, a 30-line procedure pasted into CLAUDE.md belongs in a Skill (it wastes always-on context).

6.2 Skills Essentials

- A Skill = folder with `SKILL.md` (+ optional helper files). Frontmatter: `name`, `description` (the auto-trigger text), plus optional `allowed-tools`, `disable-model-invocation` (only humans can run it - use for side-effecting skills), `user-invocable: false` (background knowledge only Claude applies), `context: fork`, and an `agent` type.
- **Progressive disclosure:** only skill descriptions load every session; the full body loads when triggered - cheap until used.
- Slash commands (`.claude/commands/*.md`) are merged into the same system - both create `/name`.

6.3 context: fork - Isolated Skill Execution (scored objective)

Add `context: fork` to a Skill's or slash command's YAML frontmatter to run it in an **isolated subagent context** instead of inline in your conversation:

```
---
name: deep-audit
description: Full dependency security audit
context: fork
allowed-tools: Read, Grep, Glob
---
Audit every dependency for known CVEs... (explicit task instructions)
```

- The skill body becomes the **prompt driving the subagent**; the fork does NOT see your conversation history.
- All intermediate work happens in the forked context - **only the result returns** to the main thread. This prevents cross-contamination of session state and keeps the main context clean.
- Default (omitted) = inline execution in the current context, with full access to conversation history.
- `context: fork` only makes sense for skills with **explicit task instructions**. A reference-only skill ('use these API conventions') forked has guidelines but no task - it returns nothing meaningful.

Situation	Frontmatter
Verbose research/audit task that would pollute the main context	<code>context: fork</code>
Skill must summarize or act on the CURRENT conversation	<code>inline</code> (no fork) - a fork cannot see the history
Conventions/guidelines Claude should apply to ongoing work	<code>inline</code> , often <code>user-invocable: false</code>
Task with side effects that only a human should trigger	<code>disable-model-invocation: true</code> (fork optional)

6.4 Plan Mode vs Direct Execution vs Multi-Phase (scored objective)

Select the agentic review/execution architecture by **task scope**, **risk level**, and **human approval requirements**:

Architecture	How it works	Choose when
Direct execution	Claude reads and edits immediately as it works	Small scope, low risk, easily reversible, trusted automation - fastest
Plan mode	Read-only research first; Claude presents a plan; edits only AFTER human approval (toggle modes with Shift+Tab)	Larger/architectural changes, risky or irreversible operations, approval required BEFORE any modification
Multi-phase workflow	Separate explore -> plan -> implement -> review phases (often separate sessions/subagents) with human gates between phases	Very large scope, high risk, multiple stakeholders/checkpoints - maximum control

REMEMBER Risk and approval scale the architecture: **low -> direct, medium/approval-first -> plan mode, large/high -> multi-phase**. 'The team must sign off before any code changes' = plan mode at minimum.

6.5 Headless Mode & CI/CD (scored objective)

`claude -p` (alias `--print`) runs Claude Code **non-interactively**: one prompt in, agent runs, result to stdout, process exits (exit code 0/1 for pipeline branching). The scored objective: configure non-interactive flags, permission modes, and **cost and turn limits that prevent runaway executions**.

```
timeout 600 claude -p "Review this diff for security issues" \
  --allowedTools "Read,Grep,Glob" \
  --max-turns 10 \
  --output-format json > review.json
```

Flag / control	Purpose
<code>-p / --print</code>	Headless single-shot mode (the entry point for all CI use)
<code>--output-format text json stream-json</code>	text for humans/scripts; json for machine parsing (includes result, session_id, cost fields); stream-json for real-time event feeds
<code>--allowedTools "Read,Grep,Glob"</code>	Pre-approve a least-privilege tool list so the run never blocks on a prompt (pair with <code>--disallowedTools</code>)
<code>--permission-mode / --dangerously-skip-permissions</code>	Permission modes: default (ask), plan (read-only), acceptEdits (auto-accept file edits), bypassPermissions (no prompts - ONLY in isolated/sandboxed environments)
<code>--max-turns N</code>	Caps agent iterations - the primary runaway-loop guard and cost cap
Budget/time guards	Cost budget caps where available (e.g. <code>--max-budget-usd</code>), monitor the cost field in JSON output, and wrap with Unix timeout
<code>--resume SESSION_ID / --continue</code>	Multi-step pipelines: capture session_id from JSON output, resume with context preserved

TRAP A CI job that 'sometimes runs forever and burns budget' is missing `--max-turns (+ timeout / budget cap)` - not missing a bigger model, and not solved by `bypassPermissions` (that removes safety, not runaway risk).

6.6 Automated Review Configuration (scored objective)

A production Claude Code review job has three requirements:

1. **Load the correct project standards** - CLAUDE.md plus the `.claude/rules/` files whose globs match the changed paths, so findings reflect THIS project's conventions.
2. **Restrict unnecessary tool access** - reviews are read-only: `--allowedTools "Read,Grep,Glob"`; no Write, no Bash. Least privilege in CI is a security control, not a style choice.
3. **Produce structured output for downstream processing** - demand a JSON schema (`{file, line, severity, category, issue, suggestion}`) via prompt or a forced tool, so the pipeline can gate merges and post PR annotations automatically.

6.7 Specialized Review Passes (scored objective)

One giant prompt asking for security AND business logic AND API design suffers **recall trade-offs from competing concerns** - attention spread thin misses issues in every category. Instead run separate focused passes, each a dedicated prompt with its OWN few-shot examples of that concern; then merge findings (and de-duplicate). Cost: more calls. Benefit: higher recall per concern - the exam-correct trade.

6.8 Reducing Review False Positives (scored objective)

Fix noisy reviews by supplying, as **persistent context applied on every review** (CLAUDE.md / rules / the review prompt template): project-specific conventions ('we use snake_case here'), accepted patterns ('this DI pattern is intentional'), and exclusion criteria ('never flag generated files, vendored code, or test fixtures'). This beats per-run prompt tweaks and beats post-filtering output, because the model stops GENERATING the noise.

6.9 High-Quality Test Generation (scored objective)

- **Provide existing test files as context** - the model imitates your real style, naming, and structure.
- **Define fixture conventions** - where fixtures/factories live and how to use them, so tests slot into the suite.
- **Specify criteria that distinguish meaningful behavioral tests from trivial assertions** - must exercise behavior, edge cases, and error paths; 'expect(true).toBe(true)' and testing-the-mock do not count.

6.10 Iterative Refinement Workflows (scored objective)

When output quality is not there yet, refine with structure - the three scored techniques:

1. **Concrete input-output examples** - show an actual input and the exactly-desired output ('given this diff, this is the review I want'). Examples beat adjectives ('be more thorough').

2. **Targeted feedback on specific failures** - name the miss and why it matters ('you missed the SQL injection on line 42 of orders.py because user input reaches the query unsanitized'), instead of vague 'do better'.
3. **Batched issue descriptions** - collect ALL known issues from a run and submit them in ONE consolidated evaluation/refinement pass, instead of drip-feeding one issue per round-trip. Fewer cycles, and the model sees the whole failure pattern at once.

REMEMBER Refinement mantra: **show, point, batch**. Show an example, point at the exact failure, batch the issues.

Module 7 - Model Context Protocol (MCP)

7.1 Architecture

MCP is an open standard connecting AI applications to external systems. A **host** application (Claude Code, Claude Desktop) runs an MCP **client** that connects to MCP **servers**; each server exposes capabilities. Claude contains no integrations itself - it **discovers** what servers advertise.

Primitive	Controlled by	Nature	Example
Tools	The MODEL (Claude decides to invoke)	Actions / parameterized operations, may have side effects	create_issue(title), merge_pr(id)
Resources	The APPLICATION (attached/read as context)	Read-only content addressed by URI; no side effects	file://schema.sql, a docs page, a config record
Prompts	The USER (explicitly selected)	Reusable prompt templates (slash-command-like)	/summarize-ticket template

7.2 Resources vs Tools - the Scored Distinction

The exam scenario: an agent doing cross-system queries burns many **exploratory tool calls** just to look at reference content (schemas, docs, configs). The fix: expose that read-only server content as **resources**, so it can be attached directly into context - fewer round trips, better agent efficiency. Keep tools for actions and dynamic parameterized queries.

REMEMBER Read-only reference content -> Resource. Action or dynamic query -> Tool.
'Reduce exploratory tool calls' is the trigger phrase for Resources.

7.3 Integrating MCP Servers (scored objective)

Three integration decisions: **scope**, **authentication**, and **verification**.

Scope	Stored / shared	Use when
local (default)	Your machine, this project only - private	Personal or experimental servers
project	.mcp.json at repo root, committed and shared with the team (teammates approve on first use)	Team-standard servers everyone needs
user	Your machine, ALL projects	Personal utilities you want everywhere

```
claude mcp add github --scope project -- npx -y @modelcontextprotocol/server-github

// .mcp.json - authentication via environment variable expansion:
{ "mcpServers": { "github": {
  "command": "npx", "args": ["-y", "@modelcontextprotocol/server-github"],
  "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" } // never hard-code secrets
} } }
```

- **Auth:** expand secrets from environment variables (`${VAR}`, with `${VAR:-default}` supported) so credentials never live in committed config.
- **Verify tool discovery:** run `/mcp` to confirm the server connected and its tools are listed - the standard first debugging step when 'the MCP tools are missing'.
- **Precedence** when names collide: local overrides project overrides user.

`mcp.json` is a **phonebook**: it tells the client where servers are and how to start them. It does not execute tools; servers advertise tools; Claude invokes them.

7.4 Writing MCP Tool Descriptions (scored objective)

Misrouting (Claude picks the wrong tool) is almost always a description problem. A strong description states:

- **Purpose** - exactly what it does.
- **Input formats** - types, units, expected shapes ('ISO-8601 date', 'repo as owner/name').
- **Use-case boundaries** - when to use it and when NOT to.
- **Relationships to similar tools** - 'search_issues finds issues by text; use get_issue when you already have the issue number.' Explicit contrast lines are what stop semantically-similar tools from being confused.

Module 8 - Prompt Engineering Fundamentals

These techniques appear inside CCAF scenarios rather than as standalone questions - know what each is FOR.

Technique	What it is	Reach for it when
Role prompting	Set a persona/expertise in the system prompt ('You are a senior security reviewer')	Tone, domain framing, consistent perspective
Few-shot examples	2-5 worked input->output examples	Format compliance and edge-case behavior ('show, don't tell')
Chain of thought	Ask for step-by-step reasoning before the answer	Math, logic, multi-step analysis; accuracy over latency
XML tags / delimiters	Wrap parts in tags: <document>, <instructions>, <example>	Separating data from instructions; long prompts; safe handling of untrusted input
Prompt templates + variables	Reusable skeleton with {{placeholders}}	Production pipelines; consistency across thousands of runs
Long-context placement	Put big documents FIRST, instructions/question at the END; ask for supporting quotes before answering	Large-document Q&A and analysis accuracy
Prefill	Begin the assistant turn yourself ('{' or '<result>')	Skip preambles, steer format (not with extended thinking)

EXAM TIP Two placement rules worth points: (1) documents before instructions in long-context prompts; (2) instructions describing UNTRUSTED text go outside the tags that contain it, and the model is told to treat tag contents as data, not commands.

Module 9 - Master Decision Trees & Memory Tables

9.1 API & Output Decisions

```
Must the response be machine-parseable, strictly?
  yes -> tool use with JSON schema + tool_choice (any / specific tool)
  no  -> prompt formatting (prefill if you just need to skip preamble)

Bulk offline work, latency tolerant? -> Message Batches API (50% cheaper)
Interactive / step-depends-on-step? -> Messages API (sync)

Output truncates (stop_reason max_tokens)?
  -> split into smaller scoped calls, merge results (NOT giant max_tokens)
```

9.2 Agent Architecture Decisions

```
Independent subtasks, latency matters      -> parallel (fan-out / fan-in)
Breadth/coverage, heterogeneous subtasks  -> coordinator-worker
Dependent stages, reliability, checkpoints -> sequential pipeline

Delegating? default goal-oriented (adaptive);
              procedural only for compliance/runbooks

Pipeline must survive crashes -> durable checkpoints + task ledger,
                               idempotent stages, resume from ledger
```

9.3 Claude Code Decisions

```
Guidance type?
  stable project fact          -> CLAUDE.md
  fact only for some paths     -> .claude/rules/*.md + glob
  reusable procedure           -> Skill (context: fork if isolation needed)
  must ALWAYS/NEVER happen    -> hook (PreToolUse block) / permissions deny
  org-wide guarantee          -> managed settings

Execution architecture?
  low risk, small              -> direct execution
  approval before edits, risky -> plan mode
  very large, multi-gate       -> multi-phase workflow

CI headless run -> -p --allowedTools (least privilege) --max-turns
                  --output-format json (+ timeout / budget cap)
```

9.4 One-Line Memory Table (high-frequency answers)

If the scenario says...	The answer is...
stop_reason is tool_use	Execute the tool, return tool_result in the next user message
stop_reason is max_tokens	Truncated - split the task / raise cap; never parse as complete
Must always use a tool	tool_choice: any (or specific tool if known)
Independent lookups too slow	Parallel tool calls in one turn
Wrong values extracted	Validation + retry with the specific errors
Model invents missing fields	Optional + nullable + 'unknown' enum in schema

If the scenario says...	The answer is...
Large PR review truncates	Per-file / per-scope calls, merge findings
Architecture change, approval first	Plan mode
Read-only reference content over MCP	Expose as MCP Resource
Action / side effect over MCP	MCP Tool
Missing topic discovered mid-research	Coordinator spawns a new subagent (dynamic decomposition)
Sources conflict in final report	Preserve uncertainty; attribute both; do not collapse
CI run occasionally runs forever	--max-turns (+ timeout / budget cap)
Reviews flag intentional patterns	Persistent conventions + exclusion criteria in context
Skill floods main context	context: fork
Team-shared MCP server	--scope project (.mcp.json, env-var auth)
Conversation degrades as it grows	Summarize / sliding window / structured state / selective retention
Resume after interruption	Checkpointed state + inject summary + re-analyze only changed files

Module 10 - Exam Traps (the classic wrong-vs-right pairs)

Trap (tempting but WRONG)	Correct principle
'The model is instructed to always...' (confidence)	Programmatic enforcement: tool_choice, validators, hooks, permissions
Bigger context window / stuff everything in	Better decomposition and retrieval (RAG, subagents, targeted reads)
Fix ordering by improving tool descriptions	Workflow/programmatic enforcement of prerequisites
Raise max_tokens to avoid truncation	Split into scoped calls and merge
Random sampling for QA review	Confidence- and characteristic-based routing
One mega review prompt for all concerns	Specialized passes with dedicated few-shot examples
Fixed pre-planned subtask sequence	Dynamic decomposition that adapts to findings
Keep pipeline state in the context window	Durable external checkpoints (files/DB) + ledger
'Never do X' in CLAUDE.md as a guarantee	PreToolUse hook / permissions deny (deterministic)
bypassPermissions to fix CI prompts anywhere	--allowedTools least privilege; bypass only in isolated sandboxes
Fine-tune for hourly-changing facts	RAG - update the knowledge base
Collapse conflicting sources into one confident claim	Preserve source-level uncertainty with attribution
Give every subagent every tool 'for flexibility'	Least-privilege tool distribution per role
Vague 'be more careful' feedback loops	Concrete examples + targeted failure feedback + batched issues
Streaming to cut cost/throughput	Streaming only improves perceived latency; Batches cut cost

EXAM TIP Meta-pattern behind most traps: the wrong answer relies on HOPE (instructions, confidence, size); the right answer relies on STRUCTURE (schemas, enforcement, decomposition, routing, isolation). When torn between two options, pick the structural one.

Module 11 - Rapid-Fire Scenario Drills (40 questions)

Answer all 40 before checking the key. Every question maps to a scored objective; the key cites the handbook section to re-read if you miss it.

Q1. An agent makes five independent price lookups per request. Users complain about latency. Best fix?

- A. Switch to the Message Batches API
- B. Have Claude emit all five tool calls in one turn and execute them concurrently
- C. Enable streaming
- D. Use a larger model

Q2. After a turn with three tool_use blocks, your code returns each tool_result in its own user message and the API errors. Why?

- A. tool_result blocks must be base64 encoded
- B. All tool_results for a turn must be in the single next user message, matching each tool_use_id
- C. You must wait 1 second between results
- D. tool_choice must be set to auto

Q3. A pipeline breaks whenever Claude replies in plain text instead of calling one of several logging tools. Guarantee a tool call:

- A. Add 'always use a tool' to the system prompt
- B. tool_choice: {type: "auto"}
- C. tool_choice: {type: "any"}
- D. Lower the temperature

Q4. Downstream code parses output of exactly one schema, produced by one known tool. Strongest configuration?

- A. tool_choice: {type: "any"}
- B. tool_choice: {type: "tool", name: "emit_report"}
- C. Few-shot examples of the JSON
- D. Prefill '{'

Q5. Classify 80,000 support tickets overnight at minimum cost; nobody is waiting on individual results.

- A. Messages API with streaming
- B. Messages API with parallel tool calls
- C. Message Batches API
- D. Fine-tune a model

Q6. A customer-facing chatbot must answer within the request. Which processing mode?

- A. Message Batches API
- B. Synchronous Messages API

- C. Cron-scheduled batches
- D. Webhook callbacks only

Q7. A whole-repo review returns JSON cut off mid-object with stop_reason max_tokens. Exam-correct fix?

- A. Set max_tokens to 500,000
- B. Ask Claude to be concise
- C. Split the review into per-file calls and merge the JSON results
- D. Retry until it fits

Q8. The extractor invents invoice numbers when documents lack them. Best fix?

- A. Raise temperature
- B. Make invoice_number optional/nullable and allow an explicit absent representation
- C. Add 'do not hallucinate' to the prompt
- D. Use a bigger model

Q9. Extraction output fails business validation (totals do not add up). Best retry design?

- A. Retry the identical prompt up to 10 times
- B. Re-prompt including the specific validation errors, bounded retries, then human review
- C. Silently correct the math in code
- D. Lower max_tokens

Q10. You can human-review only 5% of extractions. How do you pick them?

- A. Uniform random sample
- B. The first 5% each day
- C. Route by low confidence scores, poor document quality, and ambiguous/null critical fields
- D. Review only the largest documents

Q11. Your reviewer produces plausible but unreliable licensing findings. Best response?

- A. More few-shot licensing examples
- B. Define explicit exclusion boundaries so licensing findings are not generated
- C. Ask for confidence wording
- D. Increase context

Q12. Output feeds a strict parser; malformed JSON causes outages. Most reliable structured-output method?

- A. Prompt-based 'respond in JSON'
- B. Prefilled '{'
- C. Tool use with a JSON schema and forced tool_choice
- D. Stop sequences

Q13. Research must cover many heterogeneous angles of a market, adapting as findings emerge. Pattern?

- A. Sequential pipeline
- B. Coordinator-worker
- C. Single long conversation
- D. Fixed parallel fan-out only

Q14. Five truly independent data sources must be gathered as fast as possible. Pattern?

- A. Sequential pipeline
- B. Coordinator asks one worker to do all five
- C. Parallel execution with fan-in merge
- D. Batches API

Q15. Extract -> normalize -> validate -> load, where each stage needs the previous stage's output, and reliability matters most. Pattern?

- A. Parallel execution
- B. Sequential pipeline with per-stage checkpoints
- C. Coordinator-worker with no ordering
- D. One mega-prompt

Q16. A research-only subagent occasionally edits files. Best correction?

- A. Tell it not to in the prompt
- B. Restrict its tool list to Read/Grep/Glob (remove Write and Bash)
- C. Lower its temperature
- D. Give it fewer turns

Q17. A worker keeps returning to the coordinator asking for IDs and earlier findings. Root cause?

- A. The worker's model is too small
- B. Subagents share the coordinator's context, so wiring is broken
- C. The delegation prompt was not self-contained (missing findings, data, and source metadata)
- D. Too many tools

Q18. Workers return free-form essays; the synthesis agent struggles to merge and cite. Fix the worker output contract to:

- A. Prose only, but shorter
- B. Structured JSON findings + a short prose summary + per-finding citation metadata
- C. Raw tool logs
- D. Only confidence numbers

Q19. A registered subagent exists but the coordinator never delegates to it. Most likely cause?

- A. max_tokens too low

- **B.** Vague/overlapping AgentDefinition description, so it is never matched to tasks
- **C.** Streaming disabled
- **D.** Its model is set to sonnet

Q20. Mid-research, a worker uncovers an entirely new relevant subsystem. The orchestrator should:

- **A.** Ignore it to stay on plan
- **B.** Restart from scratch
- **C.** Dynamically spawn a new targeted subtask for it
- **D.** Log it for a future project

Q21. A 3-hour multi-agent pipeline crashes at 80%. Rerunning restarts from zero. What was missing?

- **A.** A bigger context window
- **B.** Durable per-stage checkpoints and a task ledger enabling resume-from-incomplete
- **C.** More retries
- **D.** Parallelism

Q22. Two sources give conflicting revenue figures. The synthesis agent should:

- **A.** Average them
- **B.** Pick the newer one silently
- **C.** Report both with attribution and flag the conflict as unresolved
- **D.** Drop the metric

Q23. An agent session degrades; inspection shows the context is 80% raw verbose tool outputs. Best technique?

- **A.** Bigger context window
- **B.** Selective retention: extract the needed facts, drop the raw dumps
- **C.** More tools
- **D.** Higher temperature

Q24. Resuming yesterday's coding session, the agent re-reads the whole repository. Better resumption design?

- **A.** Always start fresh
- **B.** Inject saved state and re-analyze only files changed since (git diff)
- **C.** Paste yesterday's full transcript
- **D.** Disable file reads

Q25. An exploration exceeds one context window and must continue tomorrow. Standard mechanism?

- **A.** Hope the summary is enough
- **B.** Scratchpad/state files written during work, re-read at the next session
- **C.** Screenshots

- **D.** Longer max_tokens

Q26. Product prices change hourly and the bot must always quote current prices.

- **A.** Fine-tune hourly
- **B.** Bigger system prompt with all prices
- **C.** RAG / tool-based retrieval of current prices
- **D.** Ask users to check the website

Q27. Claude Code must NEVER run destructive commands against production, guaranteed. Where does this belong?

- **A.** A stern paragraph in CLAUDE.md
- **B.** A PreToolUse hook / permissions deny rule (managed settings for org-wide)
- **C.** A Skill
- **D.** The README

Q28. A convention applies only to files under src/api/. Best mechanism?

- **A.** Global CLAUDE.md entry
- **B.** A .claude/rules/ file scoped with a glob to src/api/**
- **C.** A hook
- **D.** A subagent

Q29. A 25-step release procedure is pasted into CLAUDE.md and burns context every session. Better home?

- **A.** Keep it; context is free
- **B.** A Skill invoked on demand (/release)
- **C.** settings.json
- **D.** An MCP resource

Q30. A dependency-audit skill floods the main conversation with thousands of lines of intermediate output. Fix?

- **A.** Run it less often
- **B.** Add context: fork so it runs in an isolated subagent and returns only the result
- **C.** Raise the context limit
- **D.** Pipe output to /dev/null

Q31. A large architectural refactor requires team sign-off before ANY file changes. Execution architecture?

- **A.** Direct execution
- **B.** Plan mode: read-only research, present plan, edit only after approval
- **C.** bypassPermissions
- **D.** Headless mode

Q32. A months-long migration needs explore, plan, implement, and review stages with human gates between each. Architecture?

- **A.** Direct execution
- **B.** Plan mode alone
- **C.** Multi-phase workflow with approval gates between phases
- **D.** One giant session

Q33. A nightly headless Claude Code job occasionally loops for hours and burns budget. Primary missing control?

- **A.** --output-format json
- **B.** --max-turns (plus timeout/budget cap)
- **C.** --dangerously-skip-permissions
- **D.** A faster model

Q34. Configure an automated PR review job correctly:

- **A.** All tools enabled, free-text output
- **B.** Read-only tools (Read/Grep/Glob), project standards loaded, structured JSON findings for the pipeline
- **C.** Write access so it can fix issues silently
- **D.** bypassPermissions on the shared runner

Q35. After each review you send Claude one vague complaint at a time ('be more thorough') with little improvement. Better refinement loop?

- **A.** Complain more firmly
- **B.** Concrete input-output examples + feedback naming the specific missed issues + all issues batched into one consolidated pass
- **C.** Switch models every run
- **D.** Shorter prompts

Q36. Agents repeatedly call a get_schema tool dozens of times per session just to read static table definitions. Better MCP design?

- **A.** Cache in the prompt manually
- **B.** Expose the schemas as MCP resources attached as context
- **C.** Add more schema tools
- **D.** Increase max turns

Q37. The whole team needs the same GitHub MCP server, authenticated safely. Configuration?

- **A.** --scope local with tokens pasted into config
- **B.** --scope project (.mcp.json committed) with \${GITHUB_TOKEN} env-var expansion
- **C.** --scope user with a shared password
- **D.** Each dev writes their own server

Q38. A single review prompt covering security + business logic + API design misses issues in every category. Fix?

- **A.** Longer single prompt
- **B.** Separate specialized passes, each with dedicated few-shot examples, findings merged after
- **C.** Higher temperature
- **D.** Bigger diff batches

Q39. The review bot keeps flagging your intentional DI pattern and generated files. Durable fix?

- **A.** Dismiss findings manually forever
- **B.** Persistent context: project conventions, accepted patterns, and exclusion criteria applied on every review
- **C.** Disable the reviewer
- **D.** Review only new files

Q40. You need every file that CONTAINS 'processPayment' across the repo. Which Claude Code tool?

- **A.** Glob
- **B.** Read
- **C.** Grep
- **D.** Bash rm

Module 11 - Answer Key & Explanations

Q	Ans	Why (section to review)
1	B	Independent subtasks in one turn = parallel tool calls; cuts round trips (2.3)
2	B	One user message carries all tool_results, matched by id (2.1)
3	C	any = must use some tool; prompt wording is hope, not enforcement (2.2)
4	B	Known single schema -> force the specific tool (2.2, 3.1)
5	C	Bulk + offline + cost = Batches, 50% cheaper (1.5)
6	B	Interactive, user waiting = synchronous Messages (1.5)
7	C	Split scoped calls and merge; giant max_tokens is the distractor (3.5)
8	B	Fabrication is a cornered schema; optional + nullable fixes it (3.2)
9	B	Retry WITH the specific errors, bounded, then human (3.6)
10	C	Confidence/characteristic routing beats random sampling (3.7)
11	B	Unreliable category -> exclusion boundary, stop generating it (3.4)
12	C	Strict compliance = tool use + schema + forced choice (3.1)
13	B	Coverage + heterogeneous + adaptive = coordinator-worker (4.2)
14	C	Independent + latency = parallel fan-out/fan-in (4.2)
15	B	Dependent stages + reliability = sequential pipeline (4.2)
16	B	Constrain by tool restriction (least privilege), not by asking nicely (4.5)
17	C	Subagents start empty; the delegation prompt must be self-contained (4.4)
18	B	Structured + prose + citations serves synthesis and reporting (4.6)
19	B	Selection runs on the AgentDefinition description (4.7)
20	C	Dynamic decomposition adapts to discoveries (4.8)
21	B	Durable checkpoints + ledger enable resumption without rework (4.9)
22	C	Preserve source-level uncertainty; never silently collapse (4.10)
23	B	Selective retention targets verbose tool dumps (5.2)
24	B	Inject state; re-analyze only changed files (5.4)
25	B	Scratchpad files are durable cross-session memory (5.3)
26	C	Frequently changing facts = retrieval, never fine-tuning (5.7)
27	B	Guarantees need deterministic enforcement, not instructions (6.1)
28	B	Path-scoped guidance = rules + glob (6.1)
29	B	Procedures belong in Skills, loaded on demand (6.1, 6.2)
30	B	context: fork isolates execution; only the result returns (6.3)

Q	Ans	Why (section to review)
31	B	Approval before edits = plan mode (6.4)
32	C	Very large + multiple gates = multi-phase workflow (6.4)
33	B	--max-turns is the runaway/cost guard (6.5)
34	B	Standards loaded + read-only tools + structured output (6.6)
35	B	Show, point, batch (6.10)
36	B	Static read-only content = MCP resource, fewer exploratory calls (7.2)
37	B	Project scope shares config; env expansion keeps secrets out (7.3)
38	B	Specialized passes avoid recall trade-offs (6.7)
39	B	Conventions + exclusions as persistent context (6.8)
40	C	Content search = Grep; name search = Glob (5.8)

Appendix A - Full Objective Coverage Checklist (all 37 score-report objectives)

Every objective from the official CCAF score report, mapped to the section that covers it. Use this to verify complete coverage and to jump straight to weak areas.

#	Objective (abridged)	Section
1	Parallel tool calls in a single response turn to cut latency	2.3
2	Session resumption: changed-file re-analysis + context injection	5.4
3	Self-contained subagent prompts (findings, data, source metadata)	4.4
4	Goal-oriented vs procedural delegation; coordinator visibility	4.3
5	Subagent tool restrictions, context scoping, role system prompts	4.5
6	Diagnose spawning: tool permissions, AgentDefinition, wiring	4.7
7	Dynamic task decomposition adapting to findings	4.8
8	Select plan mode vs direct execution vs multi-phase workflow	6.4
9	State persistence for resumable multi-agent pipelines	4.9
10	Evaluate coordinator-worker vs parallel vs sequential patterns	4.2
11	Headless CI/CD flags, permission modes, cost/turn limits	6.5
12	Iterative refinement: examples, targeted feedback, batched issues	6.10
13	Review configs: standards loaded, tools restricted, structured output	6.6
14	context: fork for isolated Skill / slash-command execution	6.3
15	Test generation: existing tests, fixtures, behavioral criteria	6.9
16	Choose CLAUDE.md vs rules+globs vs Skills vs hooks vs permissions	6.1
17	Systematic codebase exploration with Grep/Glob/Read	5.8
18	Subagent output schemas: structured + prose + citations	4.6
19	Synthesis preserving source-level uncertainty	4.10
20	Subagent isolation, scratchpad files, targeted file reading	5.3
21	Context optimization: summarize, slide, structure, select	5.2
22	Human review routing by confidence, characteristics, ambiguity	3.7
23	Synchronous Messages vs asynchronous Message Batches	1.5
24	Specialized review passes with dedicated few-shot examples	6.7
25	Select structured output method by strictness	3.1
26	Extraction schemas: optional, nullable, enums vs fabrication	3.2
27	Tool use with JSON schemas; tool_choice to guarantee invocation	2.2 / 3.1

#	Objective (abridged)	Section
28	Resolve truncation by splitting scoped calls and merging	3.5
29	Extraction accuracy: schemas + normalization + few-shot	3.3
30	Reduce false positives via conventions and exclusions in context	6.8
31	Explicit inclusion/exclusion boundaries in prompts	3.4
32	Select the right built-in tool: Grep, Glob, Read, Bash	5.8
33	Least-privilege tool distribution across subagents	4.5
34	MCP resources vs tools; expose content as resources	7.2
35	MCP integration: server scope, env-var auth, tool discovery	7.3
36	MCP tool descriptions: purpose, formats, boundaries, contrasts	7.4
37	tool_choice guarantees + sequencing prerequisite tool data	2.2 / 2.4

Appendix B - Last-Minute Cheat Sheet (read in 10 minutes)

API

- Stateless: resend full history every call. **max_tokens** caps OUTPUT; hitting it = truncation.
- stop_reasons: end_turn (done) / tool_use (execute + return tool_result) / max_tokens (truncated) / stop_sequence (hit your delimiter).
- Interactive or dependent = Messages. Bulk, offline, cost = **Batches (50% cheaper, done within 24h)**.
- Streaming = perceived latency only.

Tools & structure

- tool_choice: optional=auto, must-use-some=**any**, must-use-this=**tool+name**, forbidden=none.
- Independent calls -> **parallel tool_use blocks in ONE turn**; all tool_results in ONE next user message; disable with disable_parallel_tool_use.
- Prerequisite ordering = programmatic/workflow enforcement, never description tweaks.
- Strictness: prompt < prefill < **tool use with JSON schema**. Missing data needs optional + nullable + 'unknown' enum. Truncation = split & merge.
- Accuracy = schema + format normalization + few-shot. Unreliable category = exclusion boundary. Review routing = confidence + doc quality + ambiguity, not random.

Multi-agent

- Coverage=coordinator-worker. Speed/independent=parallel. Dependent/reliable=sequential pipeline.
- Delegate goals, not steps (procedural only for compliance). Prompts must be **self-contained** (findings + data + sources) - subagents start empty.
- Configure per role: least-privilege tools + scoped context + role system prompt (AgentDefinition: description, prompt, tools, model).
- Worker output = structured JSON + short prose + citations. Synthesis preserves conflicts with attribution.
- Adapt subtasks to findings; persist checkpoints + task ledger externally; resume from first incomplete.

Context

- Optimize: **summarize, slide, structure, select**. Externalize: subagents, scratchpad files, targeted reads.
- Resume: inject state + re-analyze only changed files. Timestamp injected facts; revalidate stale ones.
- Curated beats bigger. RAG = large/changing facts; fine-tune = stable style; neither for general knowledge.
- Glob=names, Grep=content, Read=specific file, Bash=execute.

Claude Code

- Fact=CLAUDE.md. Scoped fact=rules+glob. Procedure=Skill. Guarantee=**hook/permission** (never prose). Org-wide=managed settings.

- **context: fork** = isolated skill run, result-only return, needs explicit task instructions, no conversation history.
- Risk ladder: direct -> plan mode (approval before edits) -> multi-phase (gates between phases).
- CI: **-p --allowedTools least-privilege --max-turns N --output-format json** + timeout/budget. `bypassPermissions` only in sandboxes.
- Reviews: load standards, read-only tools, structured JSON out; specialized passes per concern; conventions + exclusions kill false positives.
- Refine: show examples, point at exact failures, batch issues.

MCP

- Tools=model-invoked actions. **Resources=app-attached read-only content** (cuts exploratory calls). Prompts=user templates.
- Scopes: local (me, this project) / project (.mcp.json, team) / user (me, everywhere). Secrets via `ENV_VAR`. Verify with `/mcp`.
- Descriptions: purpose + input formats + boundaries + contrast with similar tools.

REMEMBER The meta-rule for every hard question: choose **STRUCTURE over HOPE** - schemas, enforcement, decomposition, routing, and isolation beat instructions, confidence, and size.